MongoDB

Complete Learning Guide

The NoSQL Document Database

Topics Covered

What is MongoDB | Databases & Collections | CRUD Operations
Query Operators | Sort & Limit | Update Operators | Aggregation Pipeline

---

1 What is MongoDB?

MongoDB is a NoSQL database that stores data as flexible JSON-like documents instead of rows and columns. It is one of the most popular databases in the world and is widely used in modern web applications.

1.1 SQL vs MongoDB — Key Differences

Feature	SQL (Relational)	MongoDB (NoSQL)
Data format	Tables with rows and columns	Collections of JSON documents
Schema	Fixed — columns defined upfront	Flexible — each document can differ
Query language	SQL (SELECT, JOIN...)	MongoDB Query Language (MQL)
Relationships	Foreign keys and JOINs	Embedded documents or references
Scaling	Vertical (bigger server)	Horizontal (more servers)
Best for	Structured, relational data	Flexible, hierarchical data

1.2 Terminology — SQL vs MongoDB

SQL Term	MongoDB Equivalent
Database	Database
Table	Collection
Row / Record	Document
Column	Field
Primary Key	_id (auto-generated)
JOIN	Embedded document / $lookup
SELECT	find()
INSERT	insertOne() / insertMany()
UPDATE	updateOne() / updateMany()
DELETE	deleteOne() / deleteMany()

1.3 What Does a Document Look Like?

A document is a set of key-value pairs stored in BSON format (Binary JSON). Unlike a SQL row, it can contain nested objects and arrays inside it.

Sample MongoDB Document

```
{
  _id:        ObjectId('64a3f1b2c9d3e4f5a6b7c8d9'),
  first_name: 'Sara',
  last_name:  'Ahmed',
  city:       'Ramallah',
  grade:      'A',
  score:      92,
  enrolled:   true,
  courses:    [ 'Database Systems', 'Web Development' ],
  address: {
    street: 'Al-Irsal St',
    zip:    '00972'
  }
}
```

Key Concepts About Documents

- Every document automatically gets a unique _id field (like a primary key).
- Fields can hold strings, numbers, booleans, arrays, and nested documents.
- Documents in the same collection do NOT need to have the same fields.
- This flexibility is the main advantage of MongoDB over relational databases.

2 Databases and Collections

2.1 Database Commands

Database Commands

```
// Show all databases on the server
show dbs

// Create or switch to a database called SchoolDB
use SchoolDB

// Show which database you are currently using
db

// Delete the current database entirely
db.dropDatabase()
```

> **Note:** *The database is only created once you insert at least one document into it.*

2.2 Collection Commands

Collection Commands

```
// Show all collections in the current database
show collections

// Create a collection explicitly
db.createCollection('students')

// Delete a collection and all its documents
db.students.drop()
```

The db.collection.method() Pattern

Almost every MongoDB operation follows this structure:

`db . collectionName . method()`

db = current database | students = collection name | find() = operation

3 INSERT — Adding Documents

MongoDB provides two methods for adding documents to a collection.

3.1 insertOne() — Insert a Single Document

```
db.collection.insertOne({ document });
```

Example — insertOne()

```
db.students.insertOne({
  first_name: 'Sara',
  last_name:  'Ahmed',
  city:       'Ramallah',
  grade:      'A',
  score:      92
})
```

Response

```
// MongoDB returns:
{ acknowledged: true, insertedId: ObjectId('64a3f1b2c9d3e4f5a6b7c8d9') }
```

3.2 insertMany() — Insert Multiple Documents

```
db.collection.insertMany([ {doc1}, {doc2}, ... ]);
```

Example — insertMany()

```
db.students.insertMany([
  { first_name: 'Omar',  last_name: 'Hassan', city: 'Nablus',   grade: 'B+',
score: 85 },
  { first_name: 'Lina',  last_name: 'Khalid', city: 'Ramallah', grade: 'A',
score: 95 },
  { first_name: 'Yusuf', last_name: 'Nasser', city: 'Hebron',   grade: 'C',
score: 71 },
  { first_name: 'Hana',  last_name: 'Samir',  city: 'Nablus',   grade: 'B',
score: 80 },
  { first_name: 'Tariq', last_name: 'Jaber',  city: 'Hebron',   grade: 'A',
score: 90 },
  { first_name: 'Rima',  last_name: 'Khalil', city: 'Ramallah', grade: 'F',
score: 45 },
  { first_name: 'Faris', last_name: 'Odeh',   city: 'Nablus',   grade: 'B+',
score: 88 }
])
```

> **Note:** *You can add any fields you want. Documents in the same collection do not need identical fields.*

4 FIND — Reading Documents

find() retrieves documents from a collection. It is the MongoDB equivalent of SELECT in SQL.

4.1 Syntax

```
db.collection.find( { filter },  { projection } );
```

Parameter	Purpose	Required?
filter	Which documents to return (like WHERE in SQL)	No — omit to get all documents
projection	Which fields to show or hide (like SELECT columns in SQL)	No — default shows all fields

4.2 Find All Documents

Example — find() All

```
// Return all documents
db.students.find()

// Return only the first match
db.students.findOne({ city: 'Nablus' })
```

4.3 Find with a Filter

Example — find() with Filter

```
// Students from Ramallah
db.students.find({ city: 'Ramallah' })

// Multiple conditions — comma means AND by default
db.students.find({ city: 'Nablus', grade: 'B+' })
```

4.4 Projection — Show or Hide Specific Fields

Pass a second argument to find(). Use 1 to include a field and 0 to exclude it.

Example — Projection

```
// Show only first_name and city, hide _id
db.students.find({}, { first_name: 1, city: 1, _id: 0 })

// Show everything EXCEPT grade
db.students.find({}, { grade: 0 })

// Filter + projection combined
db.students.find(
  { city: 'Ramallah' },
  { first_name: 1, score: 1, _id: 0 }
)
```

Projection Rules

- You cannot mix 1s and 0s in the same projection (except for _id).
- _id is always shown by default. Write _id: 0 to hide it explicitly.
- Use only 1s to pick specific fields, OR only 0s to exclude specific fields.

---

5 Query Operators — Advanced Filtering

MongoDB uses special operators that begin with $ to filter documents. These go inside the filter argument of find().

5.1 Comparison Operators

Operator	Meaning	SQL Equivalent	Example
$eq	Equal to	=	{ score: { $eq: 92 } }
$ne	Not equal to	!=	{ grade: { $ne: 'F' } }
$gt	Greater than	>	{ score: { $gt: 80 } }
$gte	Greater than or equal	>=	{ score: { $gte: 90 } }
$lt	Less than	<	{ score: { $lt: 70 } }
$lte	Less than or equal	<=	{ score: { $lte: 75 } }
$in	Value is in a list	IN	{ city: { $in: ['Ramallah','Nablus'] } }

Operator	Meaning	SQL Equivalent	Example
$nin	Value NOT in a list	NOT IN	{ city: { $nin: ['Hebron'] } }

Example — Comparison Operators

```
// Students with score greater than 80
db.students.find({ score: { $gt: 80 } })

// Students with score between 75 and 90 (inclusive)
db.students.find({ score: { $gte: 75, $lte: 90 } })

// Students from Ramallah or Nablus
db.students.find({ city: { $in: ['Ramallah', 'Nablus'] } })

// Students NOT from Hebron
db.students.find({ city: { $nin: ['Hebron'] } })
```

5.2 Logical Operators

Operator	Meaning	SQL Equivalent
$and	All conditions must be true	AND
$or	At least one condition must be true	OR
$not	Inverts the condition	NOT
$nor	None of the conditions can be true	NOR

Example — Logical Operators

```
// $and — students from Ramallah with grade A
db.students.find({ $and: [ { city: 'Ramallah' }, { grade: 'A' } ] })

// Shorthand AND — commas produce the same result
db.students.find({ city: 'Ramallah', grade: 'A' })

// $or — students from Nablus OR with grade A
db.students.find({
  $or: [ { city: 'Nablus' }, { grade: 'A' } ]
})

// $nor — NOT from Ramallah AND NOT grade F
db.students.find({
  $nor: [ { city: 'Ramallah' }, { grade: 'F' } ]
})
```

5.3 Pattern Matching — $regex (LIKE equivalent)

Use $regex to search for text patterns inside a field — equivalent to LIKE in SQL.

SQL LIKE Pattern	MongoDB $regex	Meaning
LIKE 'S%'	{ $regex: /^S/ }	Starts with S
LIKE '%a'	{ $regex: /a$/ }	Ends with a
LIKE '%ar%'	{ $regex: /ar/ }	Contains ar anywhere
LIKE 'S%' (any case)	{ $regex: /^s/i }	Starts with s, case-insensitive

Example — $regex

```
// First names starting with 'S'
db.students.find({ first_name: { $regex: /^S/ } })

// Last names ending with 'id'
db.students.find({ last_name: { $regex: /id$/ } })

// Names containing 'ar' (case-insensitive)
db.students.find({ first_name: { $regex: /ar/i } })
```

5.4 $exists — Check Whether a Field Is Present

Example — $exists

```
// Find documents where the 'phone' field EXISTS
db.students.find({ phone: { $exists: true } })

// Find documents where 'phone' does NOT exist
db.students.find({ phone: { $exists: false } })
```

---

6 Sorting, Limiting and Skipping

6.1 sort() — ORDER BY equivalent

Use 1 for ascending order (A to Z, lowest to highest) and -1 for descending (Z to A, highest to lowest).

Example — sort()

```
// Sort by score, highest first
db.students.find().sort({ score: -1 })

// Sort by city A-Z, then by score highest first
db.students.find().sort({ city: 1, score: -1 })

// Filter first, then sort
db.students.find({ grade: 'A' }).sort({ score: -1 })
```

6.2 limit() — Return Only N Results

Example — limit()

```
// Top 3 students by score
db.students.find().sort({ score: -1 }).limit(3)

// First matching Nablus student
db.students.find({ city: 'Nablus' }).sort({ score: -1 }).limit(1)
```

6.3 skip() and count()

Example — skip() and count()

```
// Page 2: skip first 3, then get the next 3
db.students.find().sort({ score: -1 }).skip(3).limit(3)

// Count documents that match a filter
db.students.find({ city: 'Ramallah' }).count()

// Count all documents in the collection
db.students.countDocuments()
```

7 UPDATE — Modifying Documents

Update operations always require an update operator such as $set or $inc. Never write a plain value without an operator — it will replace the entire document.

7.1 Update Operators

Operator	What It Does	Example
$set	Set the value of a field (or add it if missing)	{ $set: { grade: 'A+' } }
$unset	Remove a field from the document	{ $unset: { phone: '' } }
$inc	Increment or decrement a numeric field	{ $inc: { score: 5 } }
$rename	Rename a field	{ $rename: { 'grad': 'grade' } }
$push	Add an element to an array field	{ $push: { courses: 'Math' } }
$pull	Remove an element from an array field	{ $pull: { courses: 'Math' } }

7.2 updateOne() — Update the First Match

```
db.collection.updateOne( { filter }, { $operator: { field: value } });
```

Example — updateOne()

```
// Change Sara's grade to 'A+'
db.students.updateOne(
  { first_name: 'Sara' },
  { $set: { grade: 'A+' } }
)

// Add 5 bonus points to a specific student
db.students.updateOne(
  { _id: ObjectId('64a3f1b2c9d3e4f5a6b7c8d9') },
  { $inc: { score: 5 } }
)

// Update multiple fields in one operation
db.students.updateOne(
  { first_name: 'Omar' },
  { $set: { grade: 'A', score: 91, city: 'Ramallah' } }
)
```

```
// Remove a field from a document
db.students.updateOne(
  { first_name: 'Rima' },
  { $unset: { phone: '' } }
)
```

7.3 updateMany() — Update All Matching Documents

Example — updateMany()

```
// Give all Hebron students 3 bonus points
db.students.updateMany(
  { city: 'Hebron' },
  { $inc: { score: 3 } }
)

// Add a 'status' field to every document in the collection
db.students.updateMany(
  {},
  { $set: { status: 'active' } }
)
```

> **Warning:** Using {} as the filter matches ALL documents. Be very careful with updateMany({}, ...).

7.4 $push and $pull — Updating Arrays

Example — $push and $pull

```
// Add 'Data Structures' to Sara's courses array
db.students.updateOne(
  { first_name: 'Sara' },
  { $push: { courses: 'Data Structures' } }
)

// Remove 'Web Development' from Sara's courses array
db.students.updateOne(
  { first_name: 'Sara' },
  { $pull: { courses: 'Web Development' } }
)

// Add multiple items at once using $each
db.students.updateOne(
  { first_name: 'Sara' },
  { $push: { courses: { $each: ['Algorithms', 'Networks'] } } }
```

```
)
```

8 DELETE — Removing Documents

8.1 deleteOne() — Remove First Matching Document

Example — deleteOne()

```
// Delete the first student with grade F
db.students.deleteOne({ grade: 'F' })

// Delete by _id — safest since _id is always unique
db.students.deleteOne({ _id: ObjectId('64a3f1b2c9d3e4f5a6b7c8d9') })
```

8.2 deleteMany() — Remove All Matching Documents

Example — deleteMany()

```
// Delete all students with score below 50
db.students.deleteMany({ score: { $lt: 50 } })

// Delete all students from Hebron
db.students.deleteMany({ city: 'Hebron' })

// Delete ALL documents (collection remains, but is now empty)
db.students.deleteMany({})
```

deleteMany vs drop vs dropDatabase

- deleteMany({}) — removes all documents, keeps the collection structure.
- db.students.drop() — removes the entire collection (documents + structure).
- db.dropDatabase() — removes the entire database with all its collections.

9 Aggregation Pipeline

The aggregation pipeline processes documents through a series of stages. Each stage transforms the data and passes the result to the next stage. It is MongoDB's equivalent of GROUP BY, HAVING, SUM, AVG, and COUNT in SQL.

9.1 Syntax

```
db.collection.aggregate([  { $stage1: { ... } },  { $stage2: { ... }
},  ...]);
```

9.2 Pipeline Stages

Stage	SQL Equivalent	What It Does
$match	WHERE	Filter documents — always put this first for best performance
$group	GROUP BY	Group documents and apply accumulators (sum, avg, count...)
$sort	ORDER BY	Sort the documents
$project	SELECT cols	Include, exclude, or rename fields
$limit	LIMIT	Return only the first N documents
$skip	OFFSET	Skip the first N documents
$count	COUNT(*)	Count the documents and return a number
$lookup	JOIN	Join documents from another collection
$unwind	—	Flatten an array field into separate documents

9.3 $match — Filter Documents

Aggregation — $match

```
db.students.aggregate([
  { $match: { score: { $gt: 80 } } }
])

// Multiple conditions
db.students.aggregate([
  { $match: { city: 'Ramallah', grade: { $ne: 'F' } } }
])
```

9.4 $group — Group and Aggregate

$group requires an _id field that defines what to group by. Use null to group all documents together.

Accumulator	SQL Equivalent	Example Usage
$sum	SUM / COUNT	{ total: { $sum: 1 } } — counts rows
$avg	AVG	{ avg: { $avg: '$score' } }
$max	MAX	{ top: { $max: '$score' } }
$min	MIN	{ low: { $min: '$score' } }
$push	—	{ names: { $push: '$first_name' } } — builds array
$first	—	{ first: { $first: '$first_name' } }

Aggregation — $group

```
// Count students per city
db.students.aggregate([
  { $group: {
      _id:   '$city',
      total: { $sum: 1 }
  }}
])

// Average and highest score per city
db.students.aggregate([
  { $group: {
      _id:       '$city',
      avg_score: { $avg: '$score' },
      top_score: { $max: '$score' }
  }}
])

// Grand total across all students
db.students.aggregate([
  { $group: {
      _id:         null,
      total_score: { $sum: '$score' },
      count:       { $sum: 1 }
  }}
])
```

9.5 Full Multi-Stage Pipeline Example

This single pipeline does: filter failing students → group by city → keep avg >= 80 → sort descending.

SQL equivalent: SELECT city, COUNT(*), AVG(score) FROM students WHERE grade != 'F' GROUP BY city HAVING AVG(score) >= 80 ORDER BY avg_score DESC

Full Pipeline Example

```
db.students.aggregate([

  // Stage 1: filter failing students  (like WHERE)
  { $match: { grade: { $ne: 'F' } } },

  // Stage 2: group by city and compute stats  (like GROUP BY)
  { $group: {
      _id:       '$city',
      total:     { $sum: 1 },
      avg_score: { $avg: '$score' },
      top_score: { $max: '$score' }
  }},

  // Stage 3: keep only cities with avg >= 80  (like HAVING)
  { $match: { avg_score: { $gte: 80 } } },


  // Stage 4: sort by average score descending  (like ORDER BY)
  { $sort: { avg_score: -1 } }

])
```

Output

```
// Example output:
[
  { _id: 'Ramallah', total: 2, avg_score: 91.5, top_score: 95 },
  { _id: 'Hebron',   total: 1, avg_score: 90.0, top_score: 90 },
  { _id: 'Nablus',   total: 2, avg_score: 86.5, top_score: 88 }
]
```

9.6 $project — Reshape the Output

Aggregation — $project

```
// Show only name and score, rename score to 'points'
db.students.aggregate([
  { $project: {
      _id:        0,
```

```
      first_name: 1,
      points:     '$score'
  }}
])
```

10 Querying Nested Documents and Arrays

MongoDB documents can contain nested objects and arrays. Use dot notation to query inside them.

10.1 Querying Nested Fields — Dot Notation

Example — Dot Notation

```
// Document structure:
// { first_name: 'Sara', address: { city: 'Ramallah', country: 'Palestine' } }

// Query a nested field — wrap the path in quotes
db.students.find({ 'address.city': 'Ramallah' })
db.students.find({ 'address.country': 'Palestine' })
```

10.2 Querying Array Fields

Example — Array Queries

```
// Document structure:
// { first_name: 'Sara', courses: ['Database Systems', 'Web Development'] }

// Find students enrolled in 'Database Systems'
db.students.find({ courses: 'Database Systems' })

// Find students enrolled in BOTH courses  ($all)
db.students.find({ courses: { $all: ['Database Systems', 'Algorithms'] } })

// Find students with exactly 2 courses  ($size)
db.students.find({ courses: { $size: 2 } })
```

11 Quick Reference

CRUD Methods

Method	SQL Equivalent	Example
insertOne({...})	INSERT (1 row)	db.col.insertOne({ name: 'Sara' })
insertMany([...])	INSERT (many)	db.col.insertMany([{...},{...}])
find(filter, projection)	SELECT ... WHERE	db.col.find({ city: 'Nablus' })
findOne(filter)	SELECT ... LIMIT 1	db.col.findOne({ _id: id })
updateOne(filter, update)	UPDATE (1 row)	db.col.updateOne({f}, {$set:{...}})
updateMany(filter, update)	UPDATE (many)	db.col.updateMany({f}, {$set:{...}})
deleteOne(filter)	DELETE (1 row)	db.col.deleteOne({ _id: id })
deleteMany(filter)	DELETE (many)	db.col.deleteMany({ score:{$lt:50} })
aggregate([...])	GROUP BY / HAVING	db.col.aggregate([{$group:{...}}])

Operators Cheatsheet

Operator	Meaning	Operator	Meaning
$eq / $ne	Equal / Not equal	$gt / $gte	Greater than / or equal
$lt / $lte	Less than / or equal	$in / $nin	In a list / Not in a list
$and	All conditions true	$or	Any condition true
$exists	Field exists	$regex	Pattern matching
$set	Set a field value	$unset	Remove a field
$inc	Increment a number	$push / $pull	Add / remove from array

12 Practice Exercises

Use the students collection from Section 3 for all exercises. Write the full MongoDB command for each question.

Part A — Insert and Read

1. Insert a new student document with: first_name, last_name, city, grade, score, and a courses array with 2 course names.
2. Display all documents in the students collection.
3. Find students from Hebron. Show only first_name and score (hide _id).
4. Find the student with the highest score using sort() and limit().
5. Count how many students are in the collection.

Part B — Query Operators

6. Find all students with score greater than or equal to 85.
7. Find students from Ramallah or Nablus using $in.
8. Find students with score between 70 and 90 inclusive.
9. Find students whose first_name starts with 'S' using $regex.
10. Find students where grade is NOT 'F' and city is NOT 'Hebron'.

Part C — Update and Delete

11. Update one student's grade to 'A+' using their name as the filter.
12. Add 5 bonus points to all students in Nablus using $inc.
13. Add 'Algorithms' to a student's courses array using $push.
14. Add a new field 'scholarship: true' for all students with score >= 90.
15. Delete all students with score below 50.

Part D — Aggregation Pipeline

16. Count how many students are in each city.
17. Find the average score per grade.
18. Find the city with the highest average score using $group, $sort, and $limit.
19. Show only cities where average score is above 80 (use $match after $group).
20. Bonus: filter out grade-F students, group by city, compute count and average, keep groups with count >= 2, sort by average descending.

End of MongoDB Learning Guide